

Checklist

Platform Developer Experience Evaluation

1. Establish a DevEx Baseline

- Deploy a realistic demo application through the platform.
- Test the complete developer journey from code change to running application.
- Record the current developer experience before making improvements.
- Measure workflow friction and identify high-priority bottlenecks.
- Track **efficiency, satisfaction, and impact** as core Developer Experience measures.
- Capture baseline DORA metrics:
 - Deployment frequency
 - Lead time
 - Mean time to recovery (MTTR)
 - Change failure rate

2. Validate the Deployment Pipeline

- Confirm that pushing code automatically triggers the deployment workflow.
- Automatically check out the source code.
- Run automated tests.
- Run linting and other quality checks.
- Build the application automatically.
- Build a container image automatically.
- Push the image to the container registry automatically.
- Update the deployment without manual intervention.
- Verify GitOps synchronization keeps the desired and actual state aligned.
- Confirm failed deployments can be rolled back.
- Confirm that deployment actions leave an audit trail.

3. Provide Genuine Self-Service

- Allow developers to deploy without submitting tickets.
- Eliminate unnecessary manual approvals.
- Eliminate manual infrastructure configuration wherever possible.
- Provide at least one simple developer touchpoint:
 - CLI
 - UI
 - API
- Provide pre-configured templates for common application types.
- Automatically provision resources based on application requirements.
- Validate application configuration before deployment.
- Automatically generate required deployment manifests.
- Apply organizational policies automatically.
- Give developers immediate access to application endpoints after deployment.
- Give developers immediate access to monitoring information.

4. Verify Container Production Readiness

- Use a reproducible container build process.
- Use a multi-stage build where appropriate.
- Run the application as a non-root user.
- Copy only required runtime files into the production image.
- Define a health check.
- Configure resource requests.
- Configure resource limits.
- Configure liveness probes.
- Configure readiness probes.
- Prevent unnecessary privilege escalation.
- Use a read-only root filesystem where possible.
- Drop unnecessary Linux capabilities.

5. Build Security Into the Deployment

- Scan container images for known vulnerabilities/CVEs.
- Detect secrets accidentally committed to source code.
- Validate RBAC permissions.
- Apply security policies automatically.
- Verify workloads run with secure defaults.
- Configure appropriate network policies.
- Detect overly permissive access controls.
- Detect unintentionally exposed internal services.

- Automatically remediate HTTP-only access where appropriate.
- Ensure security checks happen before production deployment.

6. Test Frictionless Deployment

- Confirm a developer can deploy using minimal inputs such as:
 - Application name
 - Git repository
- Automatically create or configure the required namespace.
- Automatically build the application from source.
- Automatically configure deployment replicas.
- Provide horizontal autoscaling.
- Automatically configure ingress.
- Automatically configure HTTPS/TLS.
- Automatically configure monitoring.
- Automatically configure alerts.
- Automatically configure backups where required.
- Automatically configure security scanning.
- Automatically configure network policies.
- Automatically handle secret rotation where required.

7. Support Preview and Local Environments

- Support preview environments for production-like debugging.
- Provide automatically generated preview URLs for pull-request reviews.
- Support full-stack local development for rapid feature iteration.
- Ensure local development supports debugging and experimentation.
- Maintain parity between local and preview configurations.
- Use consistent environment-variable names across environments.
- Use consistent service-discovery patterns.
- Provide sample/initial data that works in both environments.
- Document **when** developers should use each environment.
- Document **how** developers should use each environment.
- Treat local developer setup as a first-class platform capability.
- Configure appropriate preview-environment TTLs to control costs.

8. Instrument Applications for Observability

- Capture **metrics, logs, and traces**.

- Use vendor-neutral instrumentation such as OpenTelemetry.
- Automatically collect telemetry from applications.
- Configure trace exporting.
- Configure metric exporting.
- Verify metrics are actually exported and visible.
- Capture HTTP request telemetry.
- Capture database-call telemetry.
- Add custom spans for application-specific operations.
- Add custom metrics for relevant business or application behavior.
- Record useful span attributes.
- Record exceptions in traces.
- Verify trace propagation across services.

9. Enable Log-Trace Correlation

- Use structured logging.
- Include timestamps in log entries.
- Include log levels.
- Include meaningful messages.
- Inject trace IDs into logs.
- Inject span IDs into logs.
- Verify developers can move easily from a log entry to the related trace.

10. Provide Public Application Access

- Allow developers to request public access declaratively.
- Hide ingress-controller complexity from developers.
- Hide gateway and routing complexity from developers.
- Automatically provision required ingress resources.
- Automatically create required DNS records.
- Automatically configure routing rules.
- Allow teams to specify appropriate subdomains.

11. Make HTTPS the Default

- Automatically provision TLS certificates.
- Use automated certificate management such as cert-manager.
- Integrate with an ACME-compatible certificate authority where appropriate.
- Automatically renew certificates.

- Store certificates securely.
- Monitor certificate expiration dates.
- Prevent certificate expiration from becoming a developer-facing incident.
- Make HTTPS the path of least resistance.

12. Test Developer-Facing Error Handling

- Trigger realistic deployment failures.
- Verify errors are understandable without deep Kubernetes knowledge.
- Translate infrastructure-specific errors into human-readable explanations.
- Provide remediation guidance at the point of failure.
- Avoid forcing developers to search external documentation for common errors.
- Treat error messages as documentation at the point of need.

13. Validate Recovery and Resilience

- Provide self-service rollback.
- Make rollback easily discoverable.
- Test restoration of a previous application version.
- Verify rollback does not require an operations ticket.
- Test autoscaling under a significant traffic increase.
- Confirm the application scales without manual intervention.
- Ensure production-ready defaults are enabled rather than hidden as optional features.

14. Validate Paved Paths and Templates

- Provide templates for creating new services.
- Include pre-configured CI/CD.
- Include health checks.
- Include metrics endpoints.
- Include security scanning.
- Encode organizational best practices in templates.
- Ensure developers do not have to recreate common platform patterns manually.
- Use templates to accelerate new-developer onboarding.

15. Evaluate First Impressions

- Have a developer unfamiliar with the platform complete the workflow.

- Observe the experience without providing unnecessary handholding.
- Record points of delight.
- Record points of confusion.
- Record points of friction.
- Identify steps requiring infrastructure-specific knowledge.
- Identify steps requiring another team or person.
- Identify anything requiring manual configuration.
- Verify secure defaults work automatically.
- Verify observability provides immediate value.
- Verify developers can recover safely from mistakes.

16. Re-Measure and Improve

- Repeat the same demo deployment after platform improvements.
- Recalculate workflow friction.
- Compare results with the original baseline.
- Re-measure DORA metrics.
- Re-measure developer efficiency.
- Re-measure developer satisfaction.
- Re-measure business impact.
- Prioritize the highest-friction remaining steps.
- Make improvements iteratively.
- Repeat the evaluation as the platform matures.

Final Success Criteria

- Developers can move from code to production with minimal manual effort.
- The platform balances developer velocity with governance.
- Security is built into the workflow rather than added afterward.
- Observability works by default.
- HTTPS and safe networking are automatic.
- Local and preview environments are both well supported.
- Developers can understand and recover from failures themselves.
- Platform complexity is hidden behind simple, declarative interfaces.
- Developer Experience is measured rather than assumed.
- The platform continuously reduces cognitive load and developer friction.

